Atelier HTML/CSS — Mise en page responsive et moderne

Par Robert DIASSÉ

Niveau : L1 Web

Duree estimee : 3h30

Objectifs

A la fin de cet atelier, vous serez capable de :

- Utiliser Flexbox pour aligner et distribuer des elements sur un axe
- Utiliser CSS Grid pour construire des mises en page a deux dimensions
- Appliquer `minmax()`, `auto-fit` et `clamp()` pour une grille fluide sans media query
- Ecrire des media queries pour adapter l'affichage a chaque taille d'ecran
- Livrer une page web moderne, complete et entierement responsive

---

Prerequis

- Bases HTML (balises semantiques : `header`, `nav`, `main`, `section`, `footer`)
- Notions CSS de base (selecteurs, proprietes, box model)
- Visual Studio Code installe

Mise en place

Creez un dossier `atelier-responsive/` contenant deux fichiers :

```
atelier-responsive/
├── index.html
└── style.css
```

Contenu initial de `index.html` (le squelette sera complete etape par etape) :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">

    <!--
        La balise viewport est INDISPENSABLE pour le responsive.
        Sans elle, le navigateur mobile affiche la page en version bureau
        et la reduit, rendant tout illisible.
    -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>TechBlog — Atelier Responsive</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Le contenu sera ajoute etape par etape -->

</body>
</html>
```

Partie 1 — Flexbox

Flexbox est le modele de mise en page concu pour distribuer des elements le long d'un axe (horizontal ou vertical). Il est parfait pour les navigations, les cartes alignees cote a cote, et les centages.

Les deux acteurs de Flexbox

Le conteneur active Flexbox et controle la disposition de ses enfants :

```css
.conteneur {
    display: flex;

    /* Axe principal : row = horizontal (par defaut), column = vertical */
    flex-direction: row;

    /* Alignement sur l'axe principal */
    /* flex-start | flex-end | center | space-between | space-around | space-
evenly */
    justify-content: space-between;

    /* Alignement sur l'axe secondaire */
    /* flex-start | flex-end | center | stretch */
    align-items: center;
```

```
    /* Autorise le retour a la ligne si les elements debordent */
    flex-wrap: wrap;

    /* Espace entre les elements */
    gap: 16px;
}
```

L'element enfant peut indiquer comment il occupe l'espace :

```
.enfant {
    /* flex: grandir retrecir base */
    /* 1 1 280px = peut grandir, peut retrecir, mesure ideale = 280px */
    flex: 1 1 280px;
}
```

Etape 1.1 — Navigation flexible

Ajoutez dans le `<body>` de `index.html` :

```
<!-- En-tete avec navigation -->
<header>
    <nav class="nav-principale">
        <a href="#" class="logo">TechBlog</a>
        <ul class="nav-liens">
            <li><a href="#">Accueil</a></li>
            <li><a href="#">Articles</a></li>
            <li><a href="#">Categories</a></li>
            <li><a href="#">Contact</a></li>
        </ul>
    </nav>
</header>
```

Contenu initial de `style.css` :

```
/* --- RESET MINIMAL --- */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box; /* Les bordures et paddings sont inclus dans la taille
de l'element */
}

body {
    font-family: 'Segoe UI', Arial, sans-serif;
    color: #1a1a2e;
    background: #f8f9fa;
```

```
    line-height: 1.6;
}

/* --- NAVIGATION AVEC FLEXBOX --- */
.nav-principale {
    display: flex;
    justify-content: space-between; /* Logo a gauche, liens a droite */
    align-items: center;            /* Aligne verticalement sur le centre */
    padding: 1rem 2rem;
    background: #1a1a2e;
    position: sticky; /* La barre reste visible au defilement */
    top: 0;
    z-index: 100;
}

.logo {
    color: #e94560;
    font-size: 1.5rem;
    font-weight: bold;
    text-decoration: none;
}

/* La liste de liens est elle-meme un conteneur Flex */
.nav-liens {
    display: flex;
    list-style: none;
    gap: 2rem; /* Espacement regulier entre les liens */
}

.nav-liens a {
    color: #ffffff;
    text-decoration: none;
    font-size: 0.95rem;
    transition: color 0.3s; /* Animation douce de la couleur au survol */
}

.nav-liens a:hover {
    color: #e94560;
}
```

Point de controle : ouvrez la page dans le navigateur. Le logo doit etre a gauche et les 4 liens a droite, tous centres verticalement. Survolez les liens et verifiez que la couleur change.

Etape 1.2 — Section hero et cartes Flex

Ajoutez dans `index.html`, apres la balise `</header>` :

```
<main>

    <!-- Section hero : accroche principale -->
```

```
    <section class="hero">
        <h1>Le blog des developpeurs africains</h1>
        <p>Tutoriels, astuces et actualites tech chaque semaine.</p>
        <a href="#" class="btn-hero">Lire les articles</a>
    </section>

    <!-- Section cartes : articles recents -->
    <section class="section-cartes">
        <h2 class="titre-section">Articles recents</h2>
        <div class="grille-cartes">

            <article class="carte">
                <div class="carte-image" style="background: #e94560;"></div>
                <div class="carte-contenu">
                    <span class="categorie">HTML/CSS</span>
                    <h3>Maitriser Flexbox en 30 minutes</h3>
                    <p>Un guide pratique pour comprendre le modele de boite
flexible.</p>
                    <a href="#" class="lien-carte">Lire la suite</a>
                </div>
            </article>

            <article class="carte">
                <div class="carte-image" style="background: #0f3460;"></div>
                <div class="carte-contenu">
                    <span class="categorie">JavaScript</span>
                    <h3>Les fonctions flechees expliquees</h3>
                    <p>Tout ce que vous devez savoir sur la syntaxe arrow
function.</p>
                    <a href="#" class="lien-carte">Lire la suite</a>
                </div>
            </article>

            <article class="carte">
                <div class="carte-image" style="background: #533483;"></div>
                <div class="carte-contenu">
                    <span class="categorie">Git</span>
                    <h3>Versionner son projet avec Git</h3>
                    <p>De l'installation a votre premier commit, le guide complet.
</p>
                    <a href="#" class="lien-carte">Lire la suite</a>
                </div>
            </article>

        </div>
    </section>

</main>
```

Ajoutez dans `style.css` :

```
/* --- SECTION HERO --- */
.hero {
    display: flex;
    flex-direction: column; /* Empile les elements verticalement */
    align-items: center;    /* Centre horizontalement */
    justify-content: center;
    text-align: center;
    padding: 6rem 2rem;
    background: linear-gradient(135deg, #1a1a2e, #0f3460);
    color: white;
    min-height: 70vh; /* Occupe au moins 70% de la hauteur de la fenetre */
}

.hero h1 {
    font-size: 2.8rem;
    margin-bottom: 1rem;
    line-height: 1.2;
}

.hero p {
    font-size: 1.1rem;
    margin-bottom: 2rem;
    opacity: 0.85;
    max-width: 600px;
}

.btn-hero {
    display: inline-block;
    background: #e94560;
    color: white;
    padding: 0.9rem 2.5rem;
    border-radius: 4px;
    text-decoration: none;
    font-weight: bold;
    font-size: 1rem;
    transition: background 0.3s, transform 0.2s;
}

.btn-hero:hover {
    background: #c73652;
    transform: translateY(-2px); /* Leger deplacement vers le haut au survol */
}


/* --- SECTION CARTES --- */
.section-cartes {
    padding: 4rem 2rem;
    max-width: 1200px;
    margin: 0 auto;
}

.titre-section {
    font-size: 1.8rem;
```

```
    margin-bottom: 2rem;
    text-align: center;
}

/* Conteneur Flex pour aligner les cartes */
.grille-cartes {
    display: flex;
    flex-wrap: wrap;          /* Retour a la ligne automatique si manque de place
*/
    gap: 1.5rem;
    justify-content: center;
}

.carte {
    background: white;
    border-radius: 10px;
    overflow: hidden;         /* Cache ce qui depasse des coins arrondis */
    box-shadow: 0 2px 16px rgba(0, 0, 0, 0.10);
    flex: 1 1 280px;          /* Base de 280px, peut grandir jusqu'a max-width */
    max-width: 360px;
    display: flex;
    flex-direction: column;   /* Empile l'image et le contenu verticalement */
    transition: transform 0.3s, box-shadow 0.3s;
}

.carte:hover {
    transform: translateY(-4px);
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.15);
}

.carte-image {
    height: 160px; /* Hauteur fixe pour toutes les images */
}

.carte-contenu {
    padding: 1.4rem;
    display: flex;
    flex-direction: column;
    flex: 1; /* Le contenu prend tout l'espace disponible en hauteur */
}

.categorie {
    font-size: 0.75rem;
    text-transform: uppercase;
    color: #e94560;
    font-weight: bold;
    letter-spacing: 0.05em;
    margin-bottom: 0.5rem;
}

.carte h3 {
    font-size: 1.1rem;
    margin-bottom: 0.5rem;
    color: #1a1a2e;
```

```
}

.carte p {
    font-size: 0.9rem;
    color: #666;
    flex: 1;            /* Pousse le lien vers le bas de la carte */
    margin-bottom: 1rem;
}

.lien-carte {
    color: #e94560;
    text-decoration: none;
    font-weight: bold;
    font-size: 0.9rem;
}

.lien-carte:hover {
    text-decoration: underline;
}
```

Point de controle : sur un ecran large, les 3 cartes doivent s'afficher cote a cote. Reduisez progressivement la fenetre du navigateur : les cartes doivent descendre en dessous les unes des autres de facon automatique. C'est `flex-wrap: wrap` et `flex: 1 1 280px` qui produisent cet effet.

Partie 2 — CSS Grid

Grid est le modele de mise en page concu pour les dispositions a deux dimensions (lignes ET colonnes simultanement). Il est ideal pour les layouts de page complets.

Concept fondamental

```
.conteneur {
    display: grid;

    /* Definit les colonnes et leurs tailles */
    /* 1fr = une fraction de l'espace libre */
    grid-template-columns: 1fr 1fr 1fr; /* 3 colonnes de taille egale */

    /* Peut aussi s'ecrire avec repeat() */
    grid-template-columns: repeat(3, 1fr);

    /* Espace entre toutes les cellules */
    gap: 24px;
}
```

Un element enfant peut occuper plusieurs colonnes ou lignes :

```
.grand-element {
    grid-column: span 2; /* Cet element s'etend sur 2 colonnes */
    grid-row: span 2;    /* Cet element s'etend sur 2 lignes */
}
```

Etape 2.1 — Layout deux colonnes (contenu + sidebar)

Ajoutez dans `index.html`, apres la section cartes et avant la fermeture de `</main>` :

```
    <!-- Layout principal : contenu + barre laterale -->
    <section class="layout-principal">

        <!-- Article mis en avant -->
        <main class="contenu-principal">
            <h2 class="titre-section" style="text-align: left;">A la une</h2>
            <article class="article-une">
                <div class="article-image"></div>
                <h3>Le CSS Grid va changer votre façon de coder</h3>
                <p>Decouvrez comment Grid permet de creer des mises en page
complexes avec un code minimal, lisible et maintenable. Contrairement a Flexbox
qui travaille sur un seul axe, Grid gere les lignes et les colonnes simultanement.
</p>
                <a href="#" class="lien-carte">Lire la suite</a>
            </article>
        </main>

        <!-- Barre laterale -->
        <aside class="sidebar">

            <h3>Categories</h3>
            <ul class="liste-categories">
                <li>HTML/CSS (12)</li>
                <li>JavaScript (8)</li>
                <li>Git (5)</li>
                <li>PHP (7)</li>
                <li>Python (4)</li>
            </ul>

            <h3>Newsletter</h3>
            <p class="sidebar-texte">Recevez les nouveaux articles chaque semaine.
</p>
            <input type="email" placeholder="votre@email.com" class="input-
newsletter">
            <button class="btn-newsletter">S'abonner</button>

        </aside>

    </section>
```

Ajoutez dans `style.css` :

```
/* --- LAYOUT PRINCIPAL AVEC GRID --- */
.layout-principal {
    display: grid;
    /*
        Deux colonnes :
        - 1fr : colonne principale, prend tout l'espace disponible
        - 300px : sidebar de largeur fixe
    */
    grid-template-columns: 1fr 300px;
    gap: 2rem;
    max-width: 1200px;
    margin: 0 auto;
    padding: 2rem;
}

.article-une .article-image {
    height: 280px;
    background: linear-gradient(135deg, #0f3460, #533483);
    border-radius: 10px;
    margin-bottom: 1.5rem;
}

.article-une h3 {
    font-size: 1.5rem;
    margin-bottom: 0.75rem;
    color: #1a1a2e;
}

.article-une p {
    color: #555;
    margin-bottom: 1.5rem;
    line-height: 1.7;
}

/* --- SIDEBAR --- */
.sidebar {
    background: white;
    padding: 1.5rem;
    border-radius: 10px;
    box-shadow: 0 2px 16px rgba(0, 0, 0, 0.10);
    height: fit-content; /* La sidebar ne s'etire pas sur toute la hauteur de la
grille */
    position: sticky;
    top: 80px; /* Reste visible sous la navbar lors du defilement */
}

.sidebar h3 {
    font-size: 1rem;
    font-weight: bold;
    color: #1a1a2e;
    margin-bottom: 0.75rem;
```

```
    margin-top: 1.5rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
}

.sidebar h3:first-child {
    margin-top: 0;
}

.liste-categories {
    list-style: none;
    margin-bottom: 0.5rem;
}

.liste-categories li {
    padding: 0.5rem 0;
    border-bottom: 1px solid #f0f0f0;
    font-size: 0.9rem;
    color: #555;
    cursor: pointer;
    transition: color 0.2s;
}

.liste-categories li:hover {
    color: #e94560;
}

.sidebar-texte {
    font-size: 0.85rem;
    color: #777;
    margin-bottom: 0.75rem;
}

.input-newsletter {
    width: 100%;
    padding: 0.6rem 0.8rem;
    border: 1px solid #ddd;
    border-radius: 4px;
    margin-bottom: 0.5rem;
    font-size: 0.9rem;
    box-sizing: border-box;
}

.btn-newsletter {
    width: 100%;
    padding: 0.7rem;
    background: #e94560;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
    font-size: 0.9rem;
    transition: background 0.3s;
```

```
}

.btn-newsletter:hover {
    background: #c73652;
}
```

Point de controle : le contenu principal doit occuper la majorite de la largeur, et la sidebar doit apparaitre sur la droite avec une largeur fixe de 300px.

Partie 3 — Grid avancee : minmax() et auto-fit

Ces fonctions rendent une grille completement fluide sans ecrire la moindre media query.

Concept : repeat(auto-fit, minmax(min, max))

```
.grille-auto {
    display: grid;
    /*
        auto-fit : le navigateur calcule lui-meme combien de colonnes entrent
        minmax(250px, 1fr) : chaque colonne fait au moins 250px, au plus 1fr
        Resultat : grand ecran = 4 colonnes, ecran moyen = 2, mobile = 1
    */
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 1.5rem;
}
```

Concept : clamp(minimum, valeur-ideale, maximum)

`clamp()` definit une valeur qui s'adapte a la taille de la fenetre, avec un plancher et un plafond.

```
h1 {
    /*
        Taille ideale : 4vw (4% de la largeur de la fenetre)
        Jamais en dessous de 1.5rem (mobile)
        Jamais au dessus de 3rem (tres grand ecran)
    */
    font-size: clamp(1.5rem, 4vw, 3rem);
}
```

Etape 3.1 — Grille auto-adaptable et clamp()

Ajoutez dans `index.html`, apres la section layout-principal et avant la fermeture de `</main>` :

```
    <!-- Section grille auto-adaptable -->
    <section class="section-outils">
```

```html
        <h2 class="titre-section">Technologies maitrisees</h2>
        <div class="grille-outils">
            <div class="outil-card">HTML5</div>
            <div class="outil-card">CSS3</div>
            <div class="outil-card">JavaScript</div>
            <div class="outil-card">Bootstrap</div>
            <div class="outil-card">Git</div>
            <div class="outil-card">VS Code</div>
            <div class="outil-card">PHP</div>
            <div class="outil-card">MySQL</div>
        </div>
    </section>
```

Fermez ensuite la balise `</main>`.

Ajoutez dans `style.css` :

```css
/* --- SECTION OUTILS : GRILLE AUTO-ADAPTABLE --- */
.section-outils {
    padding: 4rem 2rem;
    background: #1a1a2e;
    color: white;
}

.section-outils .titre-section {
    color: white;
}

/* Grille qui s'adapte seule selon l'espace disponible */
.grille-outils {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 1rem;
    max-width: 900px;
    margin: 0 auto;
}

.outil-card {
    background: rgba(255, 255, 255, 0.08);
    border: 1px solid rgba(255, 255, 255, 0.15);
    border-radius: 10px;
    padding: 2rem 1rem;
    text-align: center;
    font-weight: bold;

    /* clamp() adapte la taille de police a la largeur de l'ecran */
    font-size: clamp(0.85rem, 2vw, 1.05rem);

    transition: background 0.3s, transform 0.2s;
    cursor: default;
}
```

```
.outil-card:hover {
    background: #e94560;
    transform: translateY(-3px);
}
```

Point de controle : redimensionnez lentement la fenetre du navigateur. Observez que la grille passe automatiquement de 8 colonnes a 4, puis a 2, puis a 1. Aucune media query n'est necessaire : `auto-fit` et `minmax()` s'en chargent.

Partie 4 — Media Queries

Les media queries appliquent des styles specifiques selon la taille de l'ecran. On travaille en "mobile-first" : on code d'abord pour le mobile, puis on enrichit pour les ecrans plus larges avec `min-width`.

Syntaxe mobile-first

```
/* Styles de base : s'appliquent a TOUS les ecrans (mobile d'abord) */
.element {
    font-size: 1rem;
}

/* Ecrans d'au moins 768px (tablette et plus) */
@media (min-width: 768px) {
    .element {
        font-size: 1.2rem;
    }
}

/* Ecrans d'au moins 1024px (ordinateur et plus) */
@media (min-width: 1024px) {
    .element {
        font-size: 1.4rem;
    }
}
```

Points de rupture (breakpoints) utilises dans cet atelier

Nom	Seuil	Cible
Mobile	(base, aucun seuil)	Smartphones (< 576px)
Tablette	768px	Tablettes et grands smartphones
Ordinateur	1024px	Ordinateurs portables et de bureau

Etape 4.1 — Rendre le layout responsive avec les media queries

Ajoutez a la toute fin de `style.css`, apres tous les styles existants :

```
/* ============================================================
   MEDIA QUERIES — Adaptations selon la taille de l'ecran
   ============================================================ */

/* --- TABLETTE : ecrans de 768px et moins --- */
@media (max-width: 768px) {

    /* La navbar passe en colonne */
    .nav-principale {
        flex-direction: column;
        gap: 1rem;
        padding: 1rem;
        text-align: center;
    }

    /* Les liens de navigation se mettent en colonne */
    .nav-liens {
        flex-direction: column;
        gap: 0.5rem;
        align-items: center;
    }

    /* Titre du hero plus petit pour ne pas deborder */
    .hero h1 {
        font-size: 1.8rem;
    }

    /* Le layout passe a une seule colonne */
    .layout-principal {
        grid-template-columns: 1fr;
        padding: 1rem;
    }

    /* La sidebar n'est plus sticky sur mobile */
    .sidebar {
        position: static;
    }

}


/* --- MOBILE STRICT : ecrans de 480px et moins --- */
@media (max-width: 480px) {

    .hero {
        padding: 4rem 1rem;
        min-height: 50vh;
    }

    .hero h1 {
        font-size: 1.4rem;
    }
```

```
    .section-cartes {
        padding: 2rem 1rem;
    }

    .btn-hero {
        padding: 0.8rem 1.5rem;
        font-size: 0.9rem;
    }

}


/* --- ORDINATEUR : ecrans de 1024px et plus --- */
@media (min-width: 1024px) {

    /* Hero encore plus grand sur grand ecran */
    .hero h1 {
        font-size: 3.2rem;
    }

    /* Les cartes peuvent aller jusqu'a 4 par ligne */
    .grille-cartes {
        justify-content: flex-start;
    }

}
```

Point de controle : utilisez l'outil de simulation d'appareils du navigateur (F12, puis cliquez sur l'icone de telephone) pour tester les tailles suivantes :

Taille simulee	Ce que vous devez observer
375px (iPhone)	Navigation en colonne, layout en 1 colonne
768px (iPad)	Navigation en colonne, layout en 1 colonne
1024px (ordinateur)	Navigation horizontale, layout 2 colonnes
1440px (grand ecran)	Layout 2 colonnes, hero tres grand

Etape 4.2 — Ajouter le pied de page

Ajoutez dans `index.html`, apres la balise `</main>` :

```
<!-- Pied de page -->
<footer class="footer">
    <div class="footer-grille">

        <div>
            <h4>TechBlog</h4>
```

```
            <p>Le blog des developpeurs africains. Contenu publie chaque semaine.
</p>
        </div>

        <div>
            <h4>Navigation</h4>
            <ul>
                <li><a href="#">Accueil</a></li>
                <li><a href="#">Articles</a></li>
                <li><a href="#">Categories</a></li>
                <li><a href="#">Contact</a></li>
            </ul>
        </div>

        <div>
            <h4>Contact</h4>
            <p>contact@techblog.sn</p>
            <p>Dakar, Senegal</p>
        </div>

    </div>
    <p class="footer-copyright">&copy; 2025 TechBlog. Tous droits reserves.</p>
</footer>
```

Ajoutez dans `style.css`, avant le bloc des media queries :

```
/* --- PIED DE PAGE --- */
.footer {
    background: #1a1a2e;
    color: rgba(255, 255, 255, 0.80);
    padding: 3rem 2rem 1.5rem;
    margin-top: 4rem;
}

/* Grille auto-adaptable pour les colonnes du footer */
.footer-grille {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
    gap: 2rem;
    max-width: 1200px;
    margin: 0 auto 2rem;
}

.footer h4 {
    color: #e94560;
    margin-bottom: 0.75rem;
    font-size: 0.95rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
}

.footer p {
```

```
    font-size: 0.9rem;
    line-height: 1.6;
    color: rgba(255, 255, 255, 0.65);
}

.footer ul {
    list-style: none;
}

.footer ul li {
    margin-bottom: 0.4rem;
}

.footer a {
    color: rgba(255, 255, 255, 0.65);
    text-decoration: none;
    font-size: 0.9rem;
    transition: color 0.2s;
}

.footer a:hover {
    color: #e94560;
}

.footer-copyright {
    text-align: center;
    font-size: 0.8rem;
    color: rgba(255, 255, 255, 0.35);
    border-top: 1px solid rgba(255, 255, 255, 0.10);
    padding-top: 1.5rem;
    margin-top: 0;
}
```

Point de controle final : verifiez l'ensemble de la page sur trois tailles d'ecran (mobile, tablette, ordinateur). Tous les elements doivent etre lisibles, rien ne doit deborder ou se superposer.

Recapitulatif des techniques

Technique	Quand l'utiliser
`display: flex`	Aligner des elements sur un seul axe (navigation, cartes, centrage)
`flex-wrap: wrap`	Permettre le retour a la ligne automatique
`flex: 1 1 280px`	Element flexible avec une taille de base definie
`display: grid`	Mises en page a 2 dimensions (lignes ET colonnes)
`grid-template-columns`	Definir le nombre et la largeur des colonnes
`repeat(auto-fit, minmax())`	Grille fluide et responsive sans media query
`clamp(min, val, max)`	Valeur typographique ou dimensionnelle fluide

Technique	Quand l'utiliser
`@media (max-width: x)`	Modifier les styles en dessous d'un seuil (mobile-first inverse)
`@media (min-width: x)`	Enrichir les styles au dessus d'un seuil (mobile-first)

Points d'amelioration (a realiser seul apres l'atelier)

1. Centraliser les couleurs avec des variables CSS. Ajoutez en haut de `style.css`, avant tout autre style :

```
:root {
    --couleur-fond:      #1a1a2e;
    --couleur-accent:    #e94560;
    --couleur-secondaire: #0f3460;
    --couleur-texte:     #333333;
    --rayon-bordure:     10px;
}
```

Remplacez ensuite toutes les couleurs en dur par `var(--couleur-accent)` etc. Si vous souhaitez changer la couleur principale du site, il suffira de modifier une seule ligne.

2. Ajouter un mode sombre : definissez un deuxieme jeu de variables sous le selecteur `[data-theme="dark"]` et basculez entre les deux avec JavaScript en ajoutant l'attribut au `<html>`.

3. Ajouter une section "Bento Grid" : une grille asymetrique ou certains elements occupent 2 colonnes ou 2 lignes avec `grid-column: span 2` et `grid-row: span 2`.

4. Tester l'accessibilite avec l'outil Lighthouse de Chrome (F12 > Lighthouse > Accessibility) : corriger les erreurs signalees (contrastes insuffisants, liens sans intitule, images sans `alt`).

5. Ajouter une animation d'apparition au defilement en CSS pur : les elements qui entrent dans la vue s'animent avec `@keyframes` et la propriete `animation-play-state`.